

Lecture 4: Seeing Multivariate Data

The scatterplot matrix — how to read it, how to draw it, and what to do when it stops working

STAT 481 — Multivariate Analysis

Tue, Sep 1, 2026 • 50 minutes • *with a mandatory appendix, §8*

Today's Overview

- 1. The data today.** Two files, the same 175 countries. `worldhealth2022` — the eight indicators you know from Lecture 2. And `whwide2022` — the *same countries*, measured on **52** indicators instead of eight.
- 2. The scatterplot matrix.** What it is, how to read it, and how to draw one in R in a single line.
- 3. What the picture shows that the numbers do not.** We draw one on four indicators and find three things **R** cannot report: a relationship that is *curved*, a hidden *grouping* of the units, and a single *odd country*.
- 4. Where it stops working.** We draw the same plot on all 52 indicators. It is unreadable — and §5 shows the problem is worse than “too small to see”.
- 5. One picture per country.** Everything above is about *columns*. §6 does the other margin.
- 6. No new symbol today.** Today is about the eye, and about the point at which the eye stops working.

Every section ends with two boxes: a short Ask an AI box — the prompt that gets you this section's code, and the three things to check in what comes back — and, at the end of the lecture, §8, **two more displays as required reading**.

Learning Objectives

By the end of this lecture you should be able to:

1. say what the three regions of a scatterplot matrix hold, and scan one in the right order;
2. draw a scatterplot matrix in R with `ggpairs()`, and say why `stars = FALSE` is not optional in this course;
3. name and *demonstrate* three things a picture shows that **R** cannot: curvature, grouping, and an individual odd row;
4. say what goes wrong with the same display at $p = 52$ — both the obvious problem and the one that survives having more time;
5. choose a display from p , from n , and from whether the units carry group labels, and defend the choice;
6. write a prompt that gets an AI assistant to produce this code, and list what you must check before you trust the answer.

Packages today

Everything today needs four new packages. Install them once:

```
install.packages(c("GGally", "palmerpenguins", "seriation", "aplpack"))
```

- **GGally** [NEW] — `ggpairs()`, the scatterplot matrix. This is the one function to remember from today.
- **palmerpenguins** [NEW] — 344 penguins with their species recorded; the one data set today whose groups are labelled.
- **corrplot** [known] — already yours from Lecture 2.
- **seriation**, **aplpack** [NEW] — `seriate()` and `faces()`; needed only in the appendix, §8.

Or run `source("R/setup.R")` in the course project, which tells you what is missing. Open `STAT481.Rproj` first, so the relative path `"data/whwide2022.csv"` resolves. Penguin data: Horst, Hill & Gorman, Palmer Station LTER. Indicators: World Bank WDI, 2022.

1 The data today

In this section

The question. *What are we looking at today, and how is today's file different from the one you already know?*

What we do about it. Load both files, check that they line up row for row, and print the two shapes — 175×8 and 175×52 . Everything in the rest of the lecture is one of those two tables.

First, Sunday's worksheet. Worksheet 3 ended with $\binom{8}{2} = 28$ and one sentence: *the only reason we found the right pair is that I told you which one*. Today we look at the pairs rather than being told which one.

The second file is new. `whwide2022.csv` holds the **same 175 countries, in the same row order**, measured on **52** World Bank indicators for 2022 instead of eight — the eight you know, plus education, communications, population structure, land, sectors of the economy, trade, finance and transport. Its first eight numeric columns *are* the spine, unchanged. So anything that changes between the two files is a fact about the *method*, not about the sample.

```
wh <- read.csv("data/worldhealth2022.csv", row.names = 1) # 175 x 8
w  <- read.csv("data/whwide2022.csv", stringsAsFactors = FALSE)
dim(w) # 175 countries, 4 labels + 52 cols
```

```
[1] 175 56
```

Ask an AI for this code — and check it

I have a CSV at `data/whwide2022.csv` with 175 rows (countries). Columns 1-4 are labels (iso3, country, region, income); columns 5 onward are numeric indicators. Using BASE R only (no tidyverse), read it with `read.csv()`, put the indicators in `X`, and print how many of `X`'s cells are missing, as a percentage.

Then check three things. (1) Did it use `read.csv()`, or quietly switch to `readr::read_csv()`? You asked for base R; the two return different kinds of object and the rest of your script depends on which. (2) Did it hard-code 56, or compute `ncol(w)`? A hard-coded width breaks silently the day the file changes. (3) **Run it and check a number you already know.** The answer is 6.52%. If the AI reports something else, it did not read the file you think it read.

2 What a scatterplot matrix is, and how to read one

In this section

The question. *Eight columns, twenty-eight pairs, and nobody to tell you which pair matters. How do you look at all of them at once?*

What we do about it. Put every pair in one grid — the scatterplot matrix. This section is only about *reading* one: what sits in each of its three regions, and in what order to look at them. We draw one in §3.

Definition 2.1: The scatterplot matrix

For p variables it is a $p \times p$ grid of panels. Panel (j, k) — row j first, column k second, the same order as `[j, k]` in R — shows variable j against variable k . **Three regions, three different objects:**

- **Lower triangle** — the scatterplot: variable k across, variable j up. One point per *country*. This is where the data are.
- **Diagonal** — *not* a scatterplot. Panel (j, j) would be a straight line, so the space holds a one-variable summary instead: a density curve.
- **Upper triangle** — the number r_{jk} itself: the entry of **R** that the panel opposite it draws.

Scan it in this order, not left to right like a table. It takes about twenty seconds on a 4×4 grid:

- (a) **The diagonal first.** Which variables are skewed? Which are *bimodal*? A bimodal density is the loudest signal in the whole grid: it usually means there are two kinds of unit in the table.
- (b) **Then the biggest numbers**, and the panel opposite each one. A big r with a curved cloud is a warning, not a finding; so is a big r held up by three points.
- (c) **Then the whole lower triangle at once**, looking only for points outside the cloud — and noting *which* panels they are outside in.

⚠ The diagonal is not a panel

A common first-week error: “the correlation of `life_exp` with itself is 1, so the plot is broken.” The diagonal holds a *density*, the lower triangle holds *clouds*, the upper triangle holds *numbers*. Say out loud which region you are reading before you read it.

3 Drawing one, and what it shows that R does not

In this section

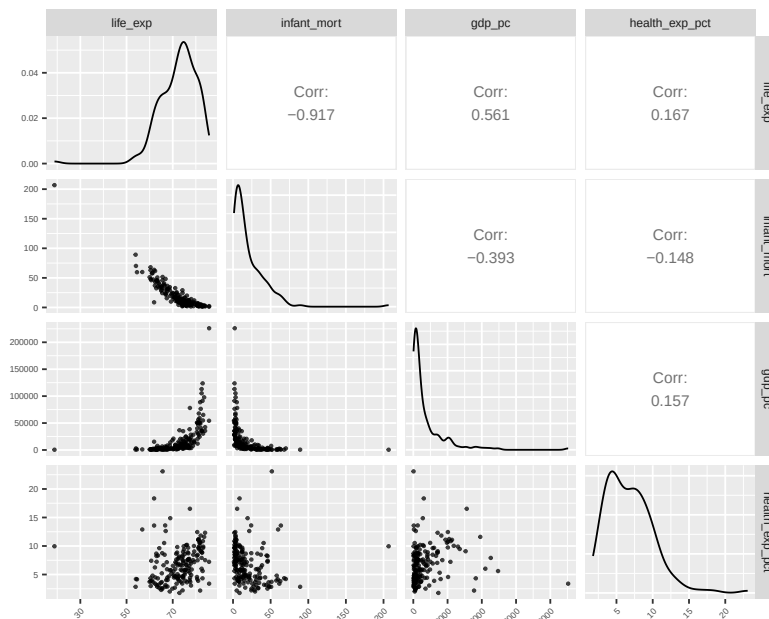
The question. Lecture 2 gave one number per pair. *What can a picture of a pair tell you that its correlation cannot?*

What we do about it. Draw the grid with one line of `ggpairs`, and hunt for the three things a number cannot carry: **curvature, grouping, one odd row**. Two of them are here; grouping needs labelled data and gets §4.

```
library(GGally)
ggpairs(wh[, c("life_exp", "infant_mort", "gdp_pc", "health_exp_pct")],
        upper = list(continuous = wrap("cor", stars = FALSE)))
```

△! One argument you must not drop

Left alone, `ggpairs()` prints `Corr: 0.561***`. The asterisks are *p*-values, and this course has no test yet. `stars = FALSE` switches them off and leaves the coefficient.



Four of the eight spine indicators, 175 countries. The full 8×8 grid is one line away — `ggpairs(wh[, 4:11])` — but 64 panels do not survive an A4 page. **Legibility beats completeness in a handout.**

Now the scan.

- (a) **The diagonal.** `infant_mort` and `gdp_pc` are violently right-skewed. `life_exp` is left-skewed with a thin tail running down to one country near 19 years. *None of the four is bimodal* — which is a finding, not an absence of one.
- (b) **Curvature** — the first thing R cannot see. The `life_exp` against `gdp_pc` panel is not a band. It rises steeply, then flattens.

```
cor(wh$life_exp, wh$gdp_pc)
cor(wh$life_exp, log(wh$gdp_pc))
```

```
[1] 0.5608026
[1] 0.7882026
```

- (c) **One odd row — the third thing.** R has no rows: it is computed *from* 175 countries and then forgets which is which. The lower triangle does not forget.

```
wh[order(wh$life_exp)[1:2], c("country", "life_exp", "infant_mort")]
wh[order(-wh$gdp_pc)[1:2], c("country", "life_exp", "gdp_pc")]
```

	country	life_exp	infant_mort
CAF	Central African Republic	18.818	206.8
SOM	Somalia, Fed. Rep.	53.931	89.2

	country	life_exp	gdp_pc
MCO	Monaco	85.746	226052.1
LUX	Luxembourg	82.946	123719.7

Fact 3.1: r can be small because the relationship is curved, not weak

Raw, $r = 0.561$. On $\log(\text{gdp_pc})$ — the same 175 countries, the same information, re-expressed so the relationship is straight — $r = 0.788$. Nothing about the world changed between those two lines. **A correlation is a statement about a straight line, and R carries no record of whether the cloud it came from was one.** The panel carries that record.

Two countries sit alone, and you found them in seconds with no formula at all. The Central African Republic has a life expectancy of 18.8 years and infant mortality of 206.8 per thousand — both the World Bank's own published 2022 values. Monaco sits at the right of every `gdp_pc` panel at \$226 052 per head. *Hold that until Thursday.*

Self-check (1 minute)

- (a) `health_exp_pct` correlates with the other three at 0.167, -0.148 and 0.157. Reason to drop the column, or to keep it?
- (b) The `life_exp-infant_mort` panel shows $r = -0.917$. Name one thing that number would look identical for, and that the panel shows is not happening.

(Answers in the summary box; try first.)

Ask an AI for this code — and check it

Using `GGally::ggpairs` in R, draw a scatterplot matrix of exactly these four columns of my data frame `wh`: `life_exp`, `infant_mort`, `gdp_pc`, `health_exp_pct`. Show the correlation in the upper panels but NOT significance stars. No tidyverse.

Then check three things. (1) Is `stars = FALSE` there? `ggpairs` prints `***` by default and an assistant will nearly always leave it. Those are p -values you have not earned yet. (2) Did you get `pairs()` instead? That is base R, it draws no correlations and no densities, and it is not what you asked for. (3) **Does the plot show *your* data?** If an assistant doubts your data exists it will often demonstrate on `iris` or `mtcars` without saying so. Four flower measurements are not four health indicators.

4 The second thing: when the groups are labelled

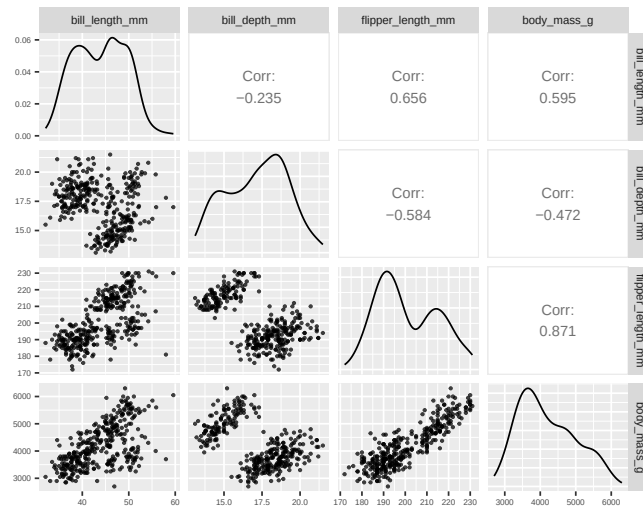
In this section

The question. *If the rows are secretly two or three different kinds of thing, does a correlation computed over all of them describe anybody?*

What we do about it. Borrow data where somebody has already done the labelling — 342 penguins of three species — compute the correlation with and without the labels, and then learn the one warning the display gives when the labels are *missing*.

`worldhealth2022` has no “kind of country” column, and almost all real data are like that. So look at data that *does* carry labels — 342 penguins, four body measurements — **with the species column set aside**:

```
library(palmerpenguins) # na.omit(): drop rows with any NA
pg <- na.omit(penguins[, c("species", "bill_length_mm", "bill_depth_mm",
                           "flipper_length_mm", "body_mass_g")])
ggpairs(pg, columns = 2:5,
        upper = list(continuous = wrap("cor", stars = FALSE)))
```



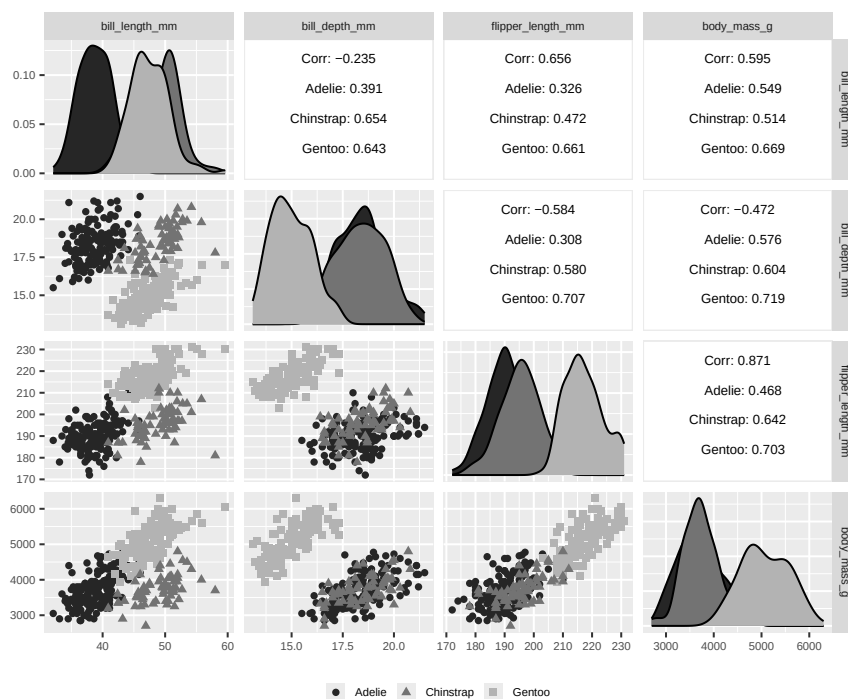
342 penguins, no labels. Read the diagonal first.

Two of the four densities are plainly bimodal — and look at `bill_depth_mm`'s left shoulder too. By rule (a): look for a grouping. There is one — switch it on, in the same call:

```
ggpairs(pg, columns = 2:5,
        mapping = aes(colour = species, shape = species),
        upper = list(continuous = wrap("cor", stars = FALSE)),
        legend = c(2, 1)) + # <- else: no legend at all
        theme(legend.position = "bottom") # aes(), theme(): ggplot2
```

△! `ggpairs` draws no legend unless you ask for one

Map a variable to colour and `ggpairs` colours the points, prints one correlation per group — and shows you **no key saying which group is which**. `legend = c(2, 1)` means *take the guide from the panel in row 2, column 1* (a scatter panel, so it carries both the shade and the shape) and show it once. **A grouped plot without a legend is not a finished figure**, and this is the commonest way to hand one in.



The legend is along the bottom: species by shape as well as grey level, so nothing is lost through a black-and-white printer. `ggpairs` now prints the pooled correlation and one per species in each upper panel. Read the `bill_depth_mm`–`flipper_length_mm` cell:

```
cor(pg$bill_depth_mm, pg$flipper_length_mm)      # all 342 pooled
sapply(split(pg, pg$species),                    # split: one piece per species
       function(d) cor(d$bill_depth_mm, d$flipper_length_mm))
```

```
[1] -0.5838512
      Adelie Chinstrap   Gentoo
0.3076202 0.5801429 0.7065634
```

Fact 4.1: Pooling groups can reverse the sign of a correlation

Over all 342 penguins, bill depth and flipper length are *negatively* related, $r = -0.584$. **Within every one of the three species they are positively related** — 0.308, 0.580, 0.707. The pooled number is not a compromise; it has the opposite sign and describes no penguin that exists. (The same reversal is in the bill cell: pooled -0.235 , within species 0.391, 0.654, 0.643.) It happens because the species differ in *both* variables at once, in opposite directions.

△! There is no “species” column in worldhealth2022

The penguins are frightening rather than interesting because somebody had already labelled them. If our 175 countries are really two kinds of country, every correlation in Lecture 2 is a pooled correlation of this type and **nothing in R will say so**. The bimodal diagonal is the one warning available — which is why we read it first, before there is a question.

Ask an AI for this code — and check it

I have a data frame `pg` with a factor column `species` and four numeric columns. In BASE R, compute the correlation of `bill_depth_mm` with `flipper_length_mm` (a) pooled over all rows and (b) separately within each species, and print both.

Then check three things. (1) `split()` + `sapply()`, or `dplyr::group_by`? Both work; only one is what you asked for, and only one runs without an install. (2) Does the per-species answer come back **named**? An unnamed vector of three numbers tells you nothing about which species is which. (3) **Trust your own result over the assistant’s commentary.** The pooled and within-group numbers here have *opposite signs*. If the AI calls that an error and “fixes” it, the AI is wrong — it is the finding.

5 The same plot on 52 indicators

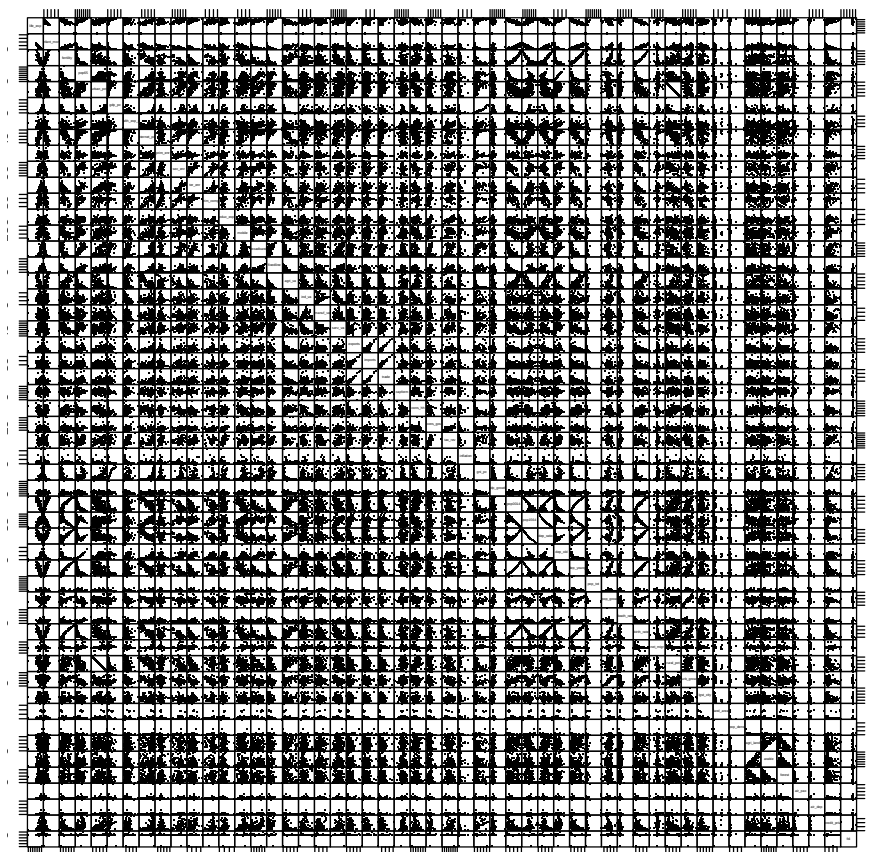
In this section

The question. It worked on four columns. *Does the same display work on fifty-two?*

What we do about it. Run it and look — that settles the obvious objection in one second. Then ask the objection nobody makes: *if the page were three metres wide and every panel were legible, would the method work?* The answer is no, and §5.1 measures why. §5.3 draws what we use instead.

Do the obvious thing first. The display worked on four columns; run it on all 52 and look at what comes back. (`pairs()` is base R’s version of the same grid — `ggpairs` on 52 columns takes minutes, and legibility is the whole question here, not styling.)

```
X <- w[, 5:56]                # the 52 indicators
pairs(X, pch = ".", gap = 0)  # same display, 52 columns wide
```

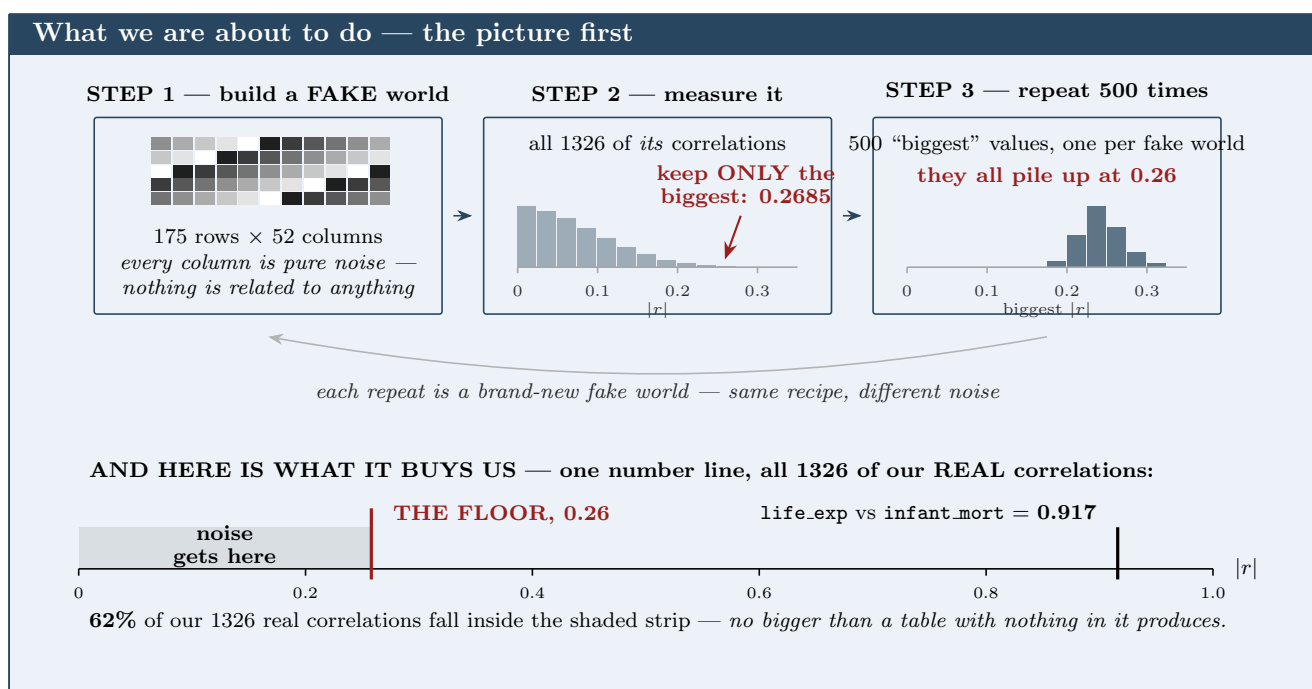


$52 \times 52 = 2704$ cells on one page. You cannot read a variable name, let alone a cloud. This is not a badly drawn figure — it is what “show me the data” returns at $p = 52$. (The shipped version sets the label size and nothing else; `R/L4_figures.R`.)

That picture needs no argument — the two triangles repeat each other, so there are $52 \times 51/2 = 1326$ distinct panels on it, and the display is finished.

But being too small to read is the *weaker* objection. Suppose the page were three metres wide and you could look at all 1326 panels properly. Would the method work then?

5.1 What the biggest correlation would be if nothing were related



What we are about to do — the picture first (continued)

- **Why bother?** Because “*I found a correlation of 0.26*” and “*the biggest of my 1326 correlations is 0.26*” are completely different claims, and the number on the page looks the same.
- **What the floor is for.** Anything below it is a correlation you would also have found in a table with nothing in it.
- **What this is *not*.** There is no hypothesis and no test anywhere in this. We are not deciding whether a correlation is “real” — **we are calibrating our own eyes**, which have no idea that a wider table produces bigger correlations for free.

Notation

Four base functions, first used here. `matrix(v, nrow = n)` folds a vector into an n -row array. `rnorm(k)` draws k independent standard normal values. `replicate(B, expr)` evaluates `expr` B times and stacks the results. `upper.tri(M)` is TRUE strictly above the diagonal, so `M[upper.tri(M)]` is the $\binom{p}{2}$ distinct entries — each pair once, no diagonal.

The four steps are four lines of R. `rnorm` is the fake world, `cor` measures it, `max(abs(...))` keeps the biggest, and `replicate` does the whole thing 500 times.

```
set.seed(481)
maxr <- replicate(500, {
  Z <- matrix(rnorm(175 * 52), nrow = 175) # 52 unrelated columns
  max(abs(cor(Z)[upper.tri(diag(52))]))    # biggest of the 1326
})
summary(maxr)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.2177	0.2465	0.2588	0.2620	0.2739	0.3741

Fact 5.1: The strongest relationship in a wide table is not evidence of one

With 175 rows and 52 *unrelated* columns, the largest of the 1326 correlations is about **0.26**, and about one run in seventeen exceeds 0.30. A single r of 0.26 on $n = 175$ would look worth reporting; *the largest of 1326* is worth nothing — and you cannot tell which you are holding by looking at it.

Why the floor rises as the table gets wider:

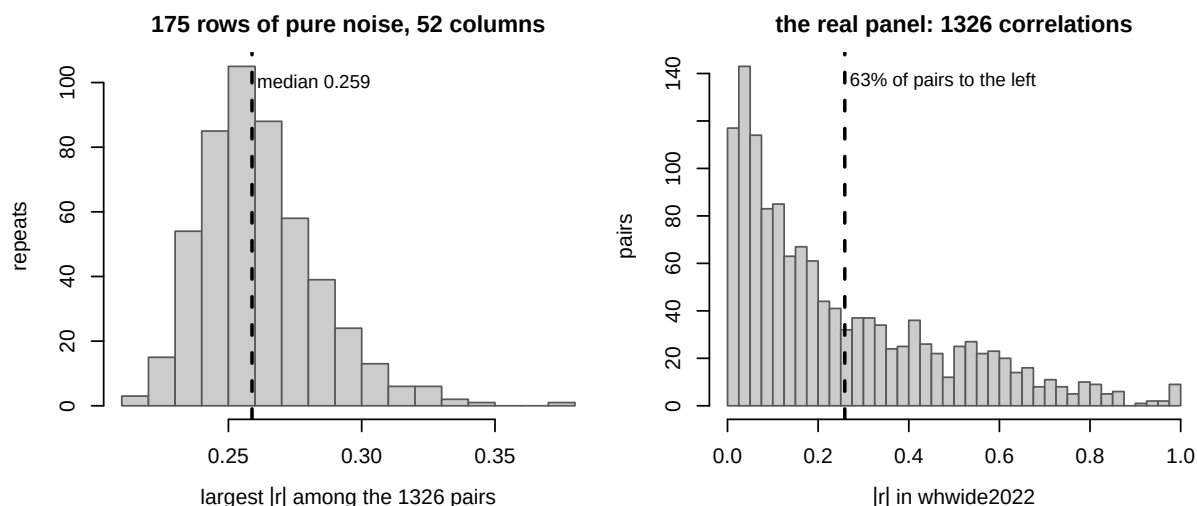
- One correlation between two unrelated columns sits near zero, but it wobbles — by about $1/\sqrt{n-1} = 0.076$ here.
- Take the biggest of many such wobbles and you are asking for the luckiest draw, so the more pairs you look at, the further out it lands.
- $p = 8$ gives 28 pairs and a floor near **0.17**; $p = 52$ gives 1326 pairs and a floor near **0.26**.

So the bar for “big” moves every time you add columns — and your eye does not move with it.

5.2 What the biggest correlations actually are

```
R <- cor(w[, 5:56], use = "pairwise.complete.obs")
rv <- abs(R[upper.tri(R)])
mean(rv <= median(maxr)) # share inside the noise floor
sum(rv > median(maxr))   # and how many survive it
```

```
[1] 0.6259427
[1] 496
```



Left: 500 largest-correlation values from unrelated columns. Right: the 1326 real ones. The dashed line is the same number in both. `mean()` of a TRUE/FALSE vector is the proportion that are TRUE.

So 496 panels clear the floor — still ten times more than anybody holds in their head. And the top of that list is worse than useless:

```
idx <- which(upper.tri(R), arr.ind = TRUE) # arr.ind: row + col
top <- data.frame(v1 = rownames(R)[idx[, 1]],
                  v2 = colnames(R)[idx[, 2]],
                  r = R[upper.tri(R)])
head(top[order(-abs(top$r)), ], 8)
```

	v1	v2	r
785	urban_pct	rural_pct	-1.0000000
1225	air_pax	air_dep	0.9923933
532	pop65	dep_old	0.9910290
528	pop1564	dep_ratio	-0.9899379
738	dep_young	birth_rate	0.9883019
592	pop0014	dep_young	0.9873288
384	gdp_pc	gni_pc	0.9816695
734	pop0014	birth_rate	0.9801812

Fact 5.2: The strongest signals in a wide table are the ones you put there

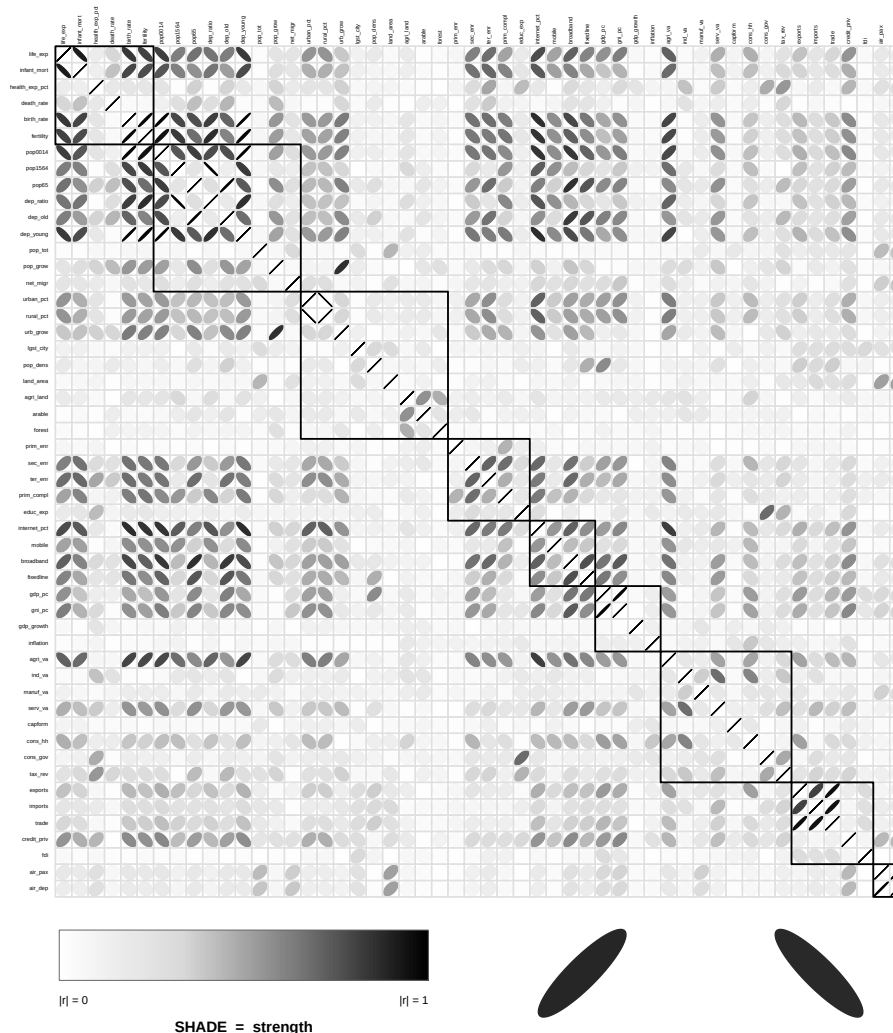
urban_pct and rural_pct correlate at exactly -1 : they sum to 100 by definition. pop65 and dep_old are the same count over two different denominators. pop1564 determines dep_ratio outright. exports + imports = trade to three decimals on all 158 countries reporting the three. Go down the ranked list. The first pair that is a *discovery* — rather than an identity we imposed, or one demographic fact restated five ways — is life_exp with infant_mort at -0.917 , number 14, and the pair Lecture 2 already found with eight columns. **Forty-four extra indicators bought nothing at the top of the list.**

5.3 What to draw instead

Give up the individual countries; keep only *which variables move together*. That means drawing **R** itself — the Lecture 2 display, now doing work it did not have to do at $p = 8$.

```
library(corrplot)
fam <- c( # the codebook's nine families, in codebook order
  "life_exp", "infant_mort", "health_exp_pct", "death_rate", "birth_rate",
  "fertility", "pop0014", "pop1564", "pop65", "dep_ratio", "dep_old",
  "dep_young", "pop_tot", "pop_grow", "net_migr", "urban_pct", "rural_pct",
  "urb_grow", "lgst_city", "pop_dens", "land_area", "agri_land", "arable",
  "forest", "prim_enr", "sec_enr", "ter_enr", "prim_compl", "educ_exp",
```

```
"internet_pct", "mobile", "broadband", "fixedline", "gdp_pc", "gni_pc",
"gdg_growth", "inflation", "agri_va", "ind_va", "manuf_va", "serv_va",
"capform", "cons_hh", "cons_gov", "tax_rev", "exports", "imports", "trade",
"credit_priv", "fdi", "air_pax", "air_dep")
corrplot(R[fam, fam], method = "ellipse", tl.cex = 0.42)
```



1326 correlations on one page, variables in **codebook family order**. Darkness is $|r|$; the tilt of the ellipse carries the sign, exactly as in Lecture 2. The printed line draws the colour version; the greyscale ramp, the family boxes and the suppressed colour bar are set in `R/L4-figures.R`.

Three readings, and the room can make all three in a minute:

- The boxes on the diagonal are dark: **variables in the same family say the same thing**, most sharply for population structure.
- One dark region spans five *different* boxes — health, population, education, communications and income all move together. That is one story told five ways.
- Sectors of the economy, land and trade are pale against everything else. Those columns carry what the first five families do not.

△! The ordering is a choice, and it is doing the work

Permute the 52 columns at random and all three readings vanish while not one correlation changes. The block structure is a property of *the order I typed*, taken from the codebook. Ordering by family is honest because the families were defined before the data were collected — but it is not the best ordering available, and finding a good one *from the data* is Lecture 21.

Self-check (1 minute)

The heat map shows all 1326 correlations on one page; the scatterplot matrix needs 1326 panels. What exactly did we give up to get that? Name two things from §§3–4. (*Answer in the summary box.*)

Ask an AI for this code — and check it

You will not remember these lines in November. You do not have to — but you do have to be able to ask for them precisely and check what comes back.

```
In base R: n = 175 rows, p = 52 columns, every column independent
standard normal. Repeat 500 times: build the matrix, compute its
correlation matrix, take the LARGEST ABSOLUTE off-diagonal
correlation. Report the median of the 500 values. Put set.seed(481)
immediately before the loop.
```

Then check three things. (1) Did it take the maximum over the **upper triangle only**? Including the diagonal gives 1 every time — the classic bug, and easy to spot because the answer comes back exactly 1. (2) Is `abs()` there? Without it you get the largest *positive* correlation, which is a different quantity. (3) **Check against a number you know: about 0.26.** If it hands you 0.05 or 1.00, one of the two above went wrong. *This is the habit that matters — never accept generated code you cannot check against something you already believe.*

6 The other margin: one picture per country

In this section

The question. Everything so far describes *columns* — the heat map has 52 columns and *no rows*, because all 175 countries vanished into **R**. *What display keeps the countries?*

What we do about it. Give every country its own small picture — a **glyph** — and read the pictures instead of the numbers. Then two follow-up questions: does it scale any better than the pairs plot did, and if the countries have to go in *some* order on the page, which order?

Definition 6.1: Glyphs

A **glyph** is **one small picture per unit** — here, one picture per country. Each variable controls one feature of that picture: the length of a ray (`stars()`), the width of a mouth (`faces()`).

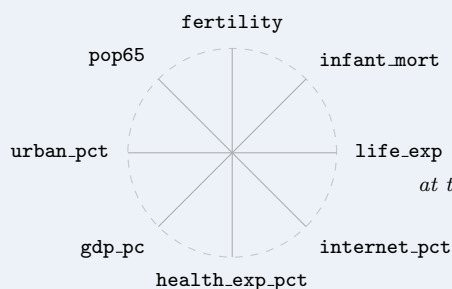
Two consequences, and both matter more than the definition:

- **One glyph carries all p variables at once.** A scatterplot panel carries two. That is the whole reason to draw one.
- **Every variable is rescaled to $[0, 1]$ across the units drawn first.** So a full-length ray means “*the largest of these eighteen*”, never an amount — and never a value you can compare with another figure.

We cannot draw 175 of them, and that is the first thing to notice. Take the 18 countries of our own region:

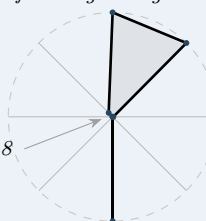
```
spine8 <- c("life_exp", "infant_mort", "fertility", "pop65",
            "urban_pct", "gdp_pc", "health_exp_pct", "internet_pct")
mena <- w$region == "Middle East, North Africa, Afghanistan & Pakistan"
M <- w[mena, spine8]
rownames(M) <- w$country[mena]
stars(M[order(rownames(M)), ], nrow = 4, ncol = 5,
      draw.segments = FALSE) # one star per country, eight rays
```

How to read a star — and what the key on the right is



THE KEY: which direction is which
(a legend, not a 20th country)

full length: highest of the 18



at the centre: lowest of the 18

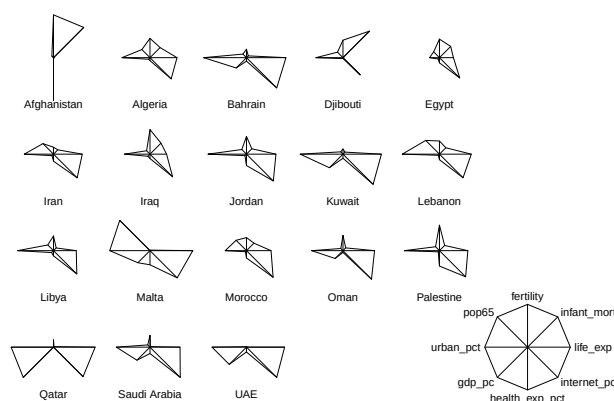
AFGHANISTAN, enlarged
(one glyph from the figure below)

- **One picture is one country.** Eighteen countries, eighteen stars; the name is printed underneath each one.
- **One *direction* is one variable** — the same variable, in the same direction, in every star. **That is all the key on the right of the figure is:** a single star with the eight directions labelled, so you can tell which spike is which. It is a legend, not a twentieth country. The first column sits at 3 o'clock and the rest follow anticlockwise.
- **The *length* of a ray is the value, rescaled.** Full length = the highest of these eighteen countries; a vertex sitting on the centre = the lowest of the eighteen; half way out = half way between the region's lowest and highest.
- **The edges are only the ray tips joined up.** `stars()` walks round the eight tips and connects them. **An edge carries no information of its own** — a long straight edge just means two *neighbouring* directions have very different values, which is why Afghanistan looks like a torn flag rather than a star.

So what do you actually look for? Three things, in this order.

- **Overall size.** A big star is a country near the top of the region on most of the eight; a small one is near the bottom on most.
- **Spikes.** A long ray with short neighbours says: extreme on *that* variable and ordinary on the rest. Read the key to name it.
- **Matching shapes.** Two countries whose stars have the same *outline* are alike on all eight at once — and that, not the size, is the comparison a scatterplot panel cannot make.

arranged by the spelling of the name



18 countries, eight rays each, in alphabetical order; the key is bottom right.

Now read two of them out loud:

- **Afghanistan** (top left) is the shape in the box above: *full length* on *fertility*, *infant_mort* and *health_exp_pct*, and *on the centre* for *life_exp*, *gdp_pc*, *urban_pct* and *internet_pct*. Highest of the eighteen on three, and lowest or all-but-lowest on four. One picture said that.
- **Qatar** is its mirror image: the rays Afghanistan has, Qatar lacks, and the reverse. **Two shapes that are opposites are two countries that are unlike each other on all eight variables at once.**

Fact 6.1: Both margins have the same wall, for the same reason

The pairs plot died at $p = 52$; the glyph display dies at $n = 175$, because comparing every picture with every other is $\binom{175}{2} = 15\,225$ comparisons and no summary is offered. **A glyph display is a display for a few dozen units, exactly as a scatterplot matrix is a display for a few variables** — the same wall, on opposite margins of the same table.

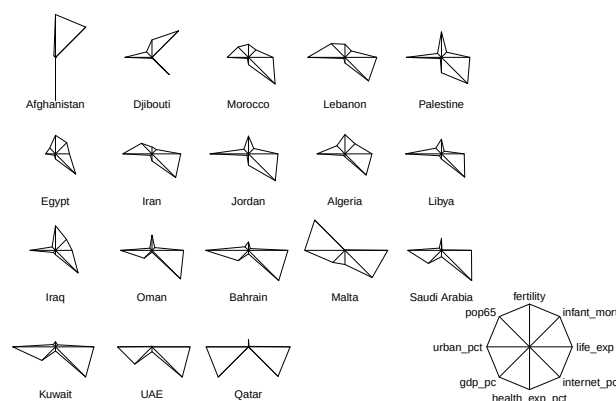
6.1 The arrangement is not part of the data

Alphabetical order is a fact about spelling. Nothing about it came from the World Bank. Use the order of one indicator instead:

```
stars(M[order(M$gdp_pc), ], nrow = 4, ncol = 5, draw.segments = FALSE)
c(n = nrow(M), p = ncol(M)); k <- nrow(M) # 18 countries, 8 cols
round(M[order(M$gdp_pc), c("gdp_pc", "pop65")][k-7:k, ], 1)
```

n	p			
18	8			
			gdp_pc	pop65
		Iraq	6520.7	3.4
		Oman	23223.6	2.7
		Bahrain	30470.5	3.5
		Malta	35658.8	19.3
		Saudi Arabia	38510.2	2.7
		Kuwait	39981.8	2.7
		United Arab Emirates	50759.8	1.6
		Qatar	88701.5	1.4

arranged by one indicator: gdp_pc, low to high



The same eighteen glyphs. Not one number was recomputed — only the order on the page changed.

Read the new picture in three bullets:

- **A gradient appears — a *rotation*, not a *growth*.** The long rays swing round the dial: *fertility* at the poor end, *internet_pct* at the rich. **They turn; they do not grow.**
- **One star breaks it, and you can see it without being told.** Malta sits 14th of 18 by income, between Bahrain and Saudi Arabia — and it carries a *full-length* *pop65* ray where both its neighbours

have a stub. The printed rows give the numbers: **19.3%** against 3.5% and 2.7%, in a top group whose pop65 runs from 1.4% to 3.5%.

- **Why it breaks:** Malta is rich *and* old, and the ordering only knows about rich. **Income order gets it wrong because income is one column and a country is eight.**

Self-check (1 minute)

So order the glyphs by something better. What would “better” mean — and what would you have to be able to *compute* before you could write the sorting line? *(Answer in the summary box.)*

Ask an AI for this code — and check it

In base R, draw one star glyph per row of an 18-row, 8-column numeric data frame M using stars(), in a 4-by-5 grid, with the row names as labels and the variable key placed so that it does NOT overlap the last glyph.

Then check three things. (1) **Look at the bottom-right corner.** stars()’s default key position sits *on top of* the last glyph. If the assistant did not pass `key.loc`, the figure is wrong in a way that is easy to miss on a projector. (2) Ask it what a ray length means. The right answer is “that variable rescaled to its own minimum and maximum *across these 18 rows*”. An assistant that says “the value” has misled you about every comparison you are about to make. (3) Ask what happens at $n = 200$. If it does not tell you the display stops working, you have found the limit of its usefulness as an adviser.

7 Choosing a display

In this section

The question. You are handed a table. *Which of today’s displays do you draw first, and what are you giving up by drawing it?*

What we do about it. One table: the display, when to use it, and what it hides — indexed by n , by p , and by whether anybody has labelled the units.

Display	Use it when	What it hides
ggpairs() scatterplot matrix	p up to about 8, any n . The default. Start here every time.	Anything that is not a property of a <i>pair</i> of variables (see the warning below). Unreadable beyond $p \approx 10$.
ggpairs(colour=) grouped version	The units carry a group label and you want to know whether the groups differ.	Nothing extra — but it is only available when somebody has done the labelling for you.
corrplot() ordered heat map	p is large and the only question left is <i>which variables move together</i> . Order the variables, and say how.	Every individual unit, all curvature, all grouping. It draws R , so it inherits every limitation of R .
stars(), faces() glyphs	n up to about 40 and the question is about a <i>unit</i> (§6). At $n = 175$, never.	Pairwise relationships entirely — and the assignment of variables to rays or to facial features is the analyst’s arbitrary choice, so permuting the columns changes which units <i>look</i> alike.
image() the data heat map	You want <i>both</i> margins at once: the standardised $n \times p$ table itself, with orders you state (§8).	Nothing, in principle — which is why it needs <i>two</i> orderings before it is readable at all.

The honest summary is short. Below $p \approx 8$ the answer is almost always the scatterplot matrix: fifteen seconds of work that prevents most of the mistakes made in the rest of this course. Above it you

must choose a margin — the heat map keeps **R** and throws the units away; the glyphs keep the units and throw the pairs away, and only up to about forty of them.

△! The pairwise fallacy — the deepest idea in this lecture

Every display in the table above shows you **pairs**. The grid on our eight spine indicators shows all $\binom{8}{2} = 28$; the heat map on 52 shows all 1326.

A unit can be unexceptional in every one of those panels and still be far from the data taken as a whole, because the information that would reveal it does not live in any pair. Worksheet 3 showed you the first step: Qatar's largest *single-column* standard score was only the **17th largest in the world** — and **one number built from two columns put it first**. Nothing on today's pages could have found that combination.

Ask an AI for this code — and check it

The most useful prompt is not “write me code”. It is “what will this hide from me”.

```
I have n = 175 rows and p = 52 numeric columns, no group labels, and
about 6% of cells missing. I want to see which variables move
together. What display should I use, what will it hide from me, and
what is the single most likely way it will mislead me? Give me the R.
```

Then check three things. (1) **Does the answer name what the display hides?** If it only sells you the plot, ask again in those words. A tool recommendation with no cost attached is not advice. (2) If it proposes a *clustered* heat map, it has quietly chosen an ordering on your behalf. Ask which ordering and why that one — you are the one who has to defend the figure. (3) Check the missing values. Every entry of `cor(X)` that involves a column with a missing value comes back NA unless you pass `use = "pairwise.complete.obs"` — on our 52 columns that is **79% of the matrix**. Code that is silent about the `use` argument has a bug you will meet three steps later.

Wrap-up

Summary of Lecture 4

- A scatterplot matrix is three objects in one grid: **densities** on the diagonal, **clouds** in the lower triangle, **numbers** in the upper. Scan diagonal → big numbers → points outside the cloud.
- **R** cannot see **curvature** ($r = 0.561$ raw against 0.788 on log **gdp-pc** — same data), **groups** (penguins: pooled -0.584 against $+0.308$, $+0.580$, $+0.707$ within species) or **individual odd rows** (CAF, Monaco). The picture sees all three.
- At $p = 52$ the same plot has $\binom{52}{2} = 1326$ panels. That is the small problem. The large one: with 52 *unrelated* columns and $n = 175$ the largest of those 1326 correlations is already about **0.26**, 62.6% of our real ones are inside that range, and six of the eight strongest are algebra we imposed.
- **Both margins have the same wall.** A pairs plot is for a few *variables*; a glyph display is for a few *units* ($n \leq 40$).
- **The R to remember:** `ggpairs()` while $p \leq 8$; `corrplot()` on an *ordered R* after that; `stars()` while $n \leq 40$. Everything else you can ask for — precisely, and then check.

Self-check answers: §3 (a) a reason to *keep* it — a column weakly related to all the others carries information the others do not. (b) -0.917 would look identical if a handful of extreme countries drove it with no relation among the rest; the panel shows a tight curved band across the whole

Summary of Lecture 4 (continued)

range, and the curve says r understates it. §5: the individual countries (all 175 rows are gone) and any curvature or grouping — the heat map is built from $\mathbf{R}$, and $\mathbf{R}$ has neither. §6: “better” means *countries alike on all eight indicators sit next to each other*, so you would first need one number saying how unlike two countries are, computed from all eight columns at once. You do not have one. That is Thursday.

The punchline. You can see 28 panels. You cannot see 1326 — and even if you could, on data with nothing in it the biggest number among them would already be about 0.26. You can see 18 glyphs; you cannot see 175. *Looking* is not a method that scales on *either* margin, and the reason is arithmetic, not stamina.

Next lecture (Thu, Sep 3): *Lecture 5 — Distance.* One number per country instead of one panel per pair — which is also the number §6 asked for and did not have. This is the last new material before Quiz 1.

Announcements:

- §8 of these notes is required reading, not an optional extra. Two more displays; read it before Sunday.
- **R Lab 1, due Sun, Sep 6.** If R, RStudio or Quarto still does not work on your machine, see me today.
- **Worksheet 4** is today’s sheet; **Worksheet 3** is now answered in full on Blackboard.
- **Practice Sheet 1: Sun, Sep 6. Sample Quiz 1** is on Blackboard; we correct it **Tue, Sep 8. Quiz 1: Thu, Sep 10**, covering Lectures 1–5.

Try at home (answers below)

Home Exercise 1. Count, then price. A colleague proposes adding 20 more indicators.

- How many panels would the scatterplot matrix have, and how many minutes at ten seconds each?
- Give a formula in p for the number of panels, and say why the grid holds only p densities however large p becomes.
- Re-run the §5 simulation with 72 columns instead of 52 (change two numbers). Does the largest correlation from unrelated columns go up or down, and by roughly how much?

Home Exercise 2. Run this. In the course project:

- Produce `ggpairs(wh[, 4:11])` — all eight spine indicators. Among the four *not* in the printed subset (`fertility`, `pop65`, `urban_pct`, `internet_pct`), report the pair with the strongest correlation and say whether its panel supports the number.
- Re-draw the §5 heat map with the columns in a random order (`set.seed(1); fam2 <- sample(fam)`). One sentence on what is lost, and one on what — if anything — changed about the data.
- Write the prompt you would give an AI assistant to produce (a), and underneath it the three things you would check in the answer. This is assessed the same way the code is.

Answers: 1) (a) $\binom{72}{2} = 2556$ panels, 426 minutes — just over seven hours. (b) $\binom{p}{2} = p(p-1)/2$; the diagonal holds one one-variable summary per variable, and there are p variables however many pairs they make. (c) Up, to roughly 0.27 — more draws from the same spread, so the largest wanders slightly further. Note how slowly it moves: the problem is not solved by measuring more carefully. 2) (a) *fertility* and *internet_pct*, $r = -0.84$, stronger than *fertility* with *pop65* (-0.67) and *urban_pct* with *internet_pct* (0.72); the panel shows a genuine downward band with no single country driving it. (b) The three block readings are gone and the page is unreadable; nothing whatever changed about the data, which is the point. (c) A good prompt names the data frame, the exact columns, the package, and the constraint (`stars = FALSE`, no tidyverse); the three checks are the ones in the §3 box.

8 Appendix — two more displays *(required reading, not optional)*

In this section

The questions. §6 left two open. *If a glyph cannot tell me which countries are alike, what can? And is there one picture that shows both margins — the countries and the indicators — at the same time?*

What we do about them. Two more displays. The first **computes the similarity first and lets it choose the arrangement** of the glyphs we already drew. The second **gives up glyphs altogether and paints the table itself**.

Read this before Sunday. Each comes with the R that makes it and an honest note on what it hides. It is also your reference sheet for the course project, where you will meet a table nobody has drawn for you.

	Column-side displays	Row-side displays
One mark is	a pair of variables	a unit — one country
Answers	which variables move together	which units are alike, and which is unlike the rest
Dies when	p is large: $\binom{p}{2}$ panels	n is large: $\binom{n}{2}$ comparisons
Seen so far	ggpairs(), corrplot()	stars(), faces()
Added here	—	seriate() (an order); and image(), which serves <i>both</i> margins at once

Setup for everything below — one extra package, **seriation**:

```
library(seriation); library(aplpack)
w <- read.csv("data/whwide2022.csv", stringsAsFactors = FALSE)
spine8 <- c("life_exp", "infant_mort", "fertility", "pop65",
            "urban_pct", "gdp_pc", "health_exp_pct", "internet_pct")
mena <- w$region == "Middle East, North Africa, Afghanistan & Pakistan"
M <- w[mena, spine8]; rownames(M) <- w$country[mena]
short <- c("Egypt, Arab Rep." = "Egypt", "Iran, Islamic Rep." = "Iran",
           "United Arab Emirates" = "UAE", "West Bank and Gaza" = "Palestine")
j <- rownames(M) %in% names(short)      # so the printed names and
rownames(M)[j] <- short[rownames(M)[j]] # the figure labels agree
c(countries = nrow(M), indicators = ncol(M))
```

```
countries indicators
      18           8
```

8.1 Glyphs in a computed order

In this section

The question. §6 left one open: income order got Malta wrong. *Can a computer choose the order for me, so that countries alike on all eight indicators end up side by side?*

What we do about it. Reverse the order of operations. Instead of drawing the pictures and hoping the similar ones catch the eye, **compute the similarity first and let it choose the arrangement**. Then redraw exactly the same eighteen glyphs in that arrangement and compare.

Definition 8.1: Seriation

Seriation chooses an order for the rows of a table so that similar rows end up next to each other.

It is a seating plan. You are given a chart saying, for every pair of guests, how badly they get on. You have one long table. Seriation finds the seating order that makes the *total* awkwardness

Definition 8.1: Seriation (continued)

between neighbours as small as possible. `seriate()` does the searching; `get_order()` hands back the seating list.

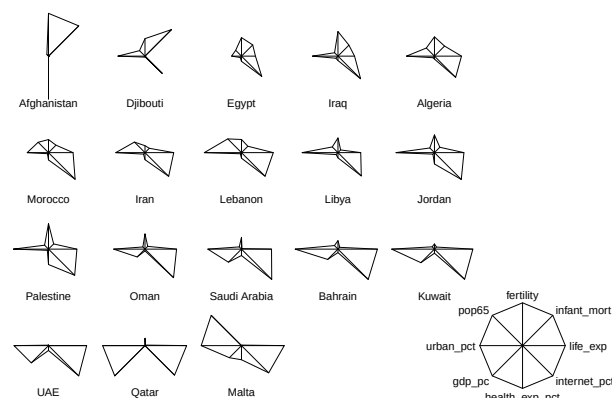
What it needs, and what it does not:

- It needs *one number per pair* saying how unlike those two rows are. **That number is the whole reason this is in an appendix and not in the lecture — it is Thursday's material.**
- It does *not* need groups, and it does not produce any. It produces an order, nothing more. Any “groups” you see are your own eyes working on that order.

```
set.seed(481) # method = "TSP" is randomised
o <- get_order(seriate(dist(scale(M)), method = "TSP",
                      control = list(rep = 20)))
if (M$gdp_pc[o[1]] > M$gdp_pc[o[length(o)]]) o <- rev(o) # low 1st
rownames(M)[o]
```

```
[1] "Afghanistan" "Djibouti" "Egypt" "Iraq"
[5] "Algeria" "Morocco" "Iran" "Lebanon"
[9] "Libya" "Jordan" "Palestine" "Oman"
[13] "Saudi Arabia" "Bahrain" "Kuwait" "UAE"
[17] "Qatar" "Malta"
```

arranged so that similar countries are adjacent



The same 18 stars, in the computed order. Against the income order: the low-income spikes still come first, but **Malta has moved past Qatar to the very end** instead of sitting stranded among the Gulf states. Nothing about the data changed.

```
stars(M[o, ], nrow = 4, ncol = 5, draw.segments = FALSE)
faces(M[o, ], face.type = 0, nrow.plot = 3, ncol.plot = 7)
# the top panel below is the SAME call on M[order(rownames(M)), ]
```

How to read a Chernoff face

Same principle as the star, different features. One face is one country, the name is printed above it, and each variable is rescaled across the eighteen exactly as before — **a bigger feature means a higher value**, full stop.

Which feature is which variable. There is no key printed on the figure, so here is the assignment `faces()` used (it is the order of the columns of `M`, nothing else):

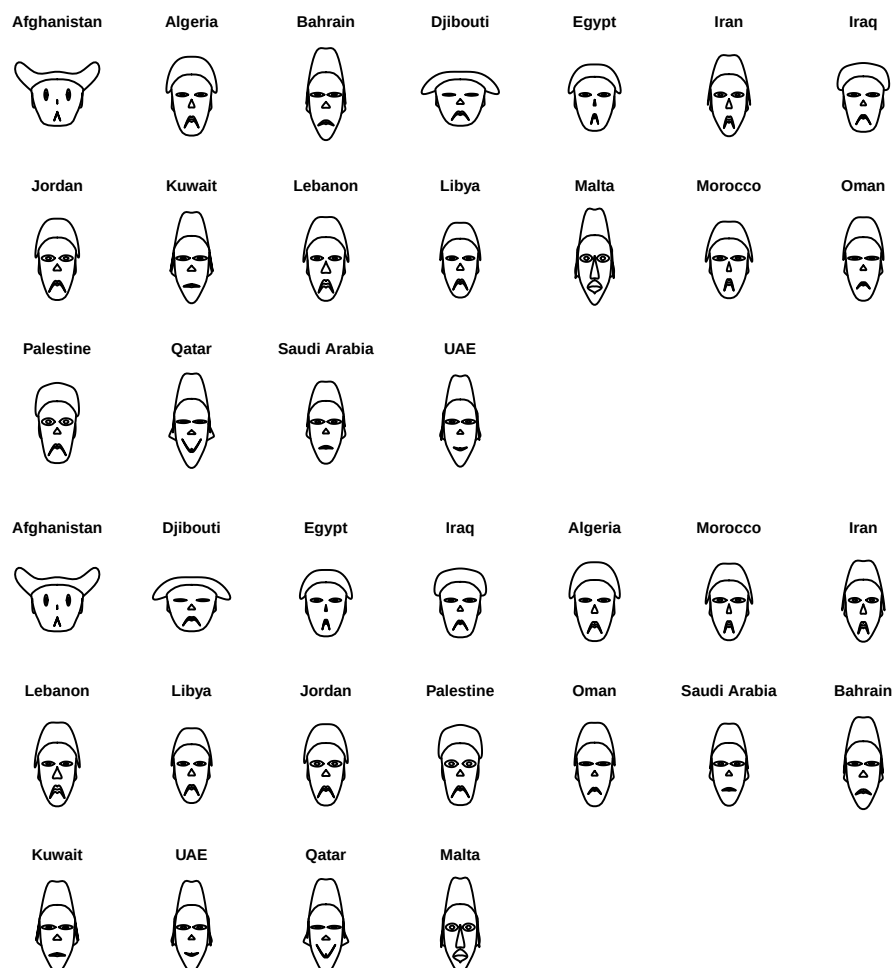
How to read a Chernoff face (continued)

Feature	Variable	Feature	Variable
height of face	life_exp	width of mouth	urban_pct
width of face	infant_mort	smiling	gdp_pc
structure of face	fertility	height of eyes	health_exp_pct
height of mouth	pop65	width of eyes	internet_pct

So, to read one country: find a feature you can actually see, look it up, say the sentence. *Afghanistan's face is short and wide* $\Rightarrow$ lowest `life_exp` and highest `infant_mort` of the eighteen. *Qatar's face is tall and narrow* $\Rightarrow$ the reverse. That is the same statement the star made with rays.

Two rules that will save you from this display:

- **Do not read a face as a mood.** Your brain reports “sad” or “friendly” before you can stop it. **There is no emotion in the data.** A “sad” face is a country with a low `gdp_pc` and nothing else, and the fact that you felt something about it is a defect of the display, not a finding.
- **Do not trust which features you noticed.** You spot ears and face width instantly and eye height never — but the plot gave the loud features and the quiet ones to variables purely by column order. **The variables you “see” are the ones that got lucky in the assignment.** The next box shows the assignment running out and starting over.



The same eighteen faces twice — alphabetical (top), computed order (bottom). Not one number differs between the two panels.

Compare the two panels in three bullets:

- **Top panel: a scramble.** Alphabetical neighbours look nothing like each other, and nothing you notice means anything.

- **Bottom panel: a progression.** Read it left to right, row by row: the wide faces and big ears at the poor end give way to a run of near-identical narrow Gulf faces, and Malta sits apart at the very end — exactly where the seriated stars put it.
- **Same eighteen faces, same eighteen countries, same numbers. The arrangement is doing all of the work** — and the arrangement came from a computed dissimilarity, not from the faces.

△! “Is there a package that draws the faces and tells me which are alike?”

No — and the reason is the point, not an accident. A face has no metric: nothing says how far “wide mouth, small eyes” is from “narrow mouth, large eyes”, so there is nothing for a package to compute. What exists is the other order of operations: *compute the similarity first, and let it choose the arrangement*, which is what `seriate()` did above.

And `faces()` makes the case against itself if you let it. Its default `print.info = TRUE` prints the assignment, and with eight variables for fifteen facial features it **runs out and starts over**: `life_exp` drives the height of the face *and* the height of the hair, `gdp_pc` the smile *and* the width of the ear. Three of your eight variables are drawn twice, in features of wildly unequal prominence — and which three depends only on the order of your columns.

```
effect of variables:
modified item      Var
"height of face"   "life_exp"
"width of face"    "infant_mort"
"structure of face" "fertility"
"height of mouth"  "pop65"
"width of mouth"   "urban_pct"
"smiling"          "gdp_pc"
"height of eyes"   "health_exp_pct"
"width of eyes"    "internet_pct"
"height of hair"   "life_exp"
"width of hair"    "infant_mort"
"style of hair"    "fertility"
"height of nose"   "pop65"
"width of nose"    "urban_pct"
"width of ear"     "gdp_pc"
"height of ear"    "health_exp_pct"
```

8.2 The data heat map — both margins at once

In this section

The question. Every display so far served one margin: pairs and **R** describe *columns*, glyphs describe *rows*. *Is there one picture that shows the countries and the indicators at the same time?*

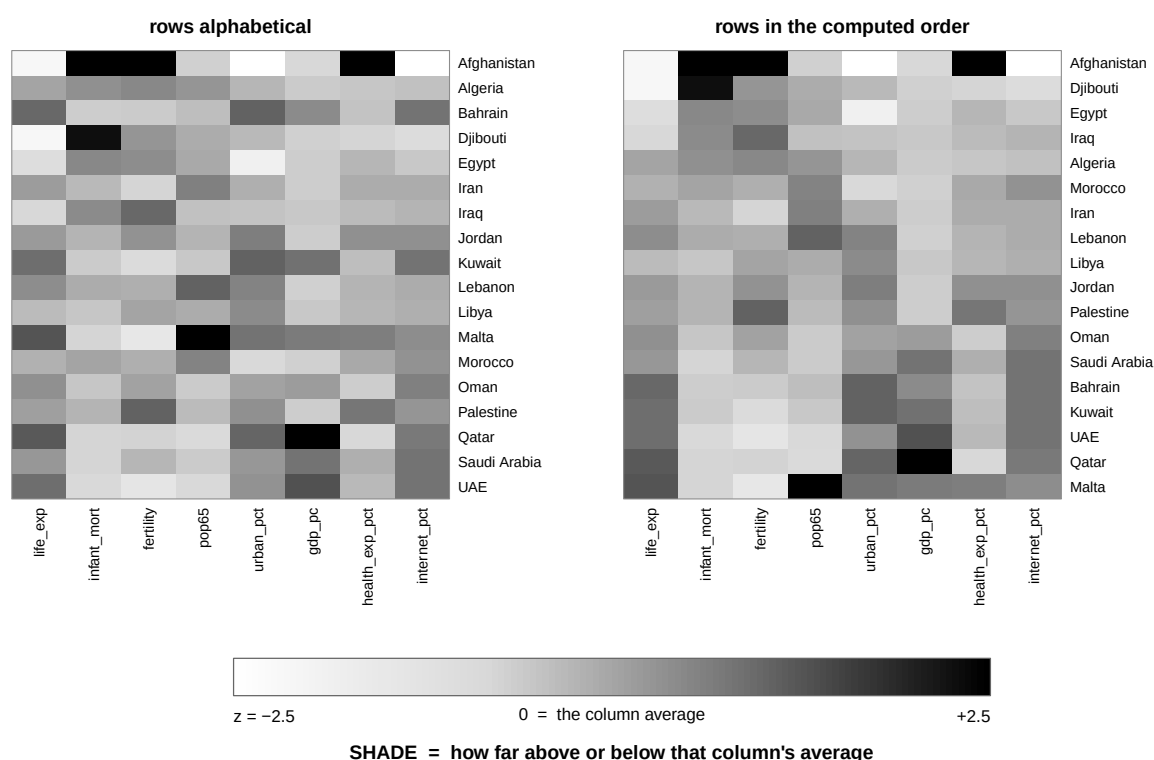
What we do about it. Stop drawing pictures *of* the table and draw the table itself — one shaded cell per measurement. We build it first on **18 countries and 8 indicators**, where every single cell is legible, and only then on the full 40×52 .

Start small: 18 countries, 8 indicators, 144 cells. The same eighteen countries as the stars, the same eight indicators.

```
Zm <- scale(M)                # z-scores: the 8 columns share a scale
Zm[Zm > 2.5] <- 2.5           # clip, or one country owns the ramp
Zm[Zm < -2.5] <- -2.5
grey <- colorRampPalette(c("white", "grey85", "grey45", "black"))(120)
A <- Zm[order(rownames(Zm)), ] # LEFT panel: countries alphabetical
image(1:8, 1:18, t(A[18:1, ]), col = grey, zlim = c(-2.5, 2.5))
# RIGHT panel: the identical line with Zm[o, ] for A -- o is the
# computed order of the previous section, and nothing else changes.
```

How to read a data heat map

- **One row is one country** (names down the right-hand side); **one column is one indicator** (names along the bottom). **One cell is one measurement** — country i 's value of indicator j .
- **The shade is the z -score, not the value.** Dark = well above that indicator's average; light = well below; mid-grey = near it. Standardising is what lets `gdp_pc` and `fertility` share one greyscale at all; `colorRampPalette()` is what builds that greyscale, in 120 steps.
- **Do not read one cell. Read a strip.** A single cell tells you one number you could have looked up. A horizontal strip that is dark on the left and light on the right is a *kind of country*, and that is the only thing this display is good for.
- **A vertical strip is the mirror image:** a column block that is uniformly dark for one set of rows and light for another is a group of indicators telling the same story — §5's heat map, seen from the other side.
- **Nothing here is legible until *both* margins are ordered.** Compare the two panels below: identical numbers, identical greyscale, and only one of them says anything.



The same 144 numbers twice. Left: countries alphabetical. Right: countries in the computed order of §8.1.

Columns identical in both. *The key under the two panels applies to both.*

Do it now on the small figure. In the right-hand panel, `life_exp` runs light at the top to dark at the bottom and `fertility` runs the other way, so the rows are graded from short lives and many births to the reverse. And the `pop65` column has **one black cell and no other** — the last row, Malta, exactly the country that broke the star display.

Fact 8.1: What the “computed order” was computed FROM

This is the question to ask of *any* reordered display, and here it has a short answer.

- **The columns were not computed at all.** They are in the codebook's family order — §5 — which is to say, the order somebody typed.
- **The rows were.** `seriate()` was handed exactly one object: `dist(scale(M))`, a table carrying one number for every *pair* of countries, saying how far apart the two are once every column has been put in z -scores. Nothing else — no labels, no groups, no names.

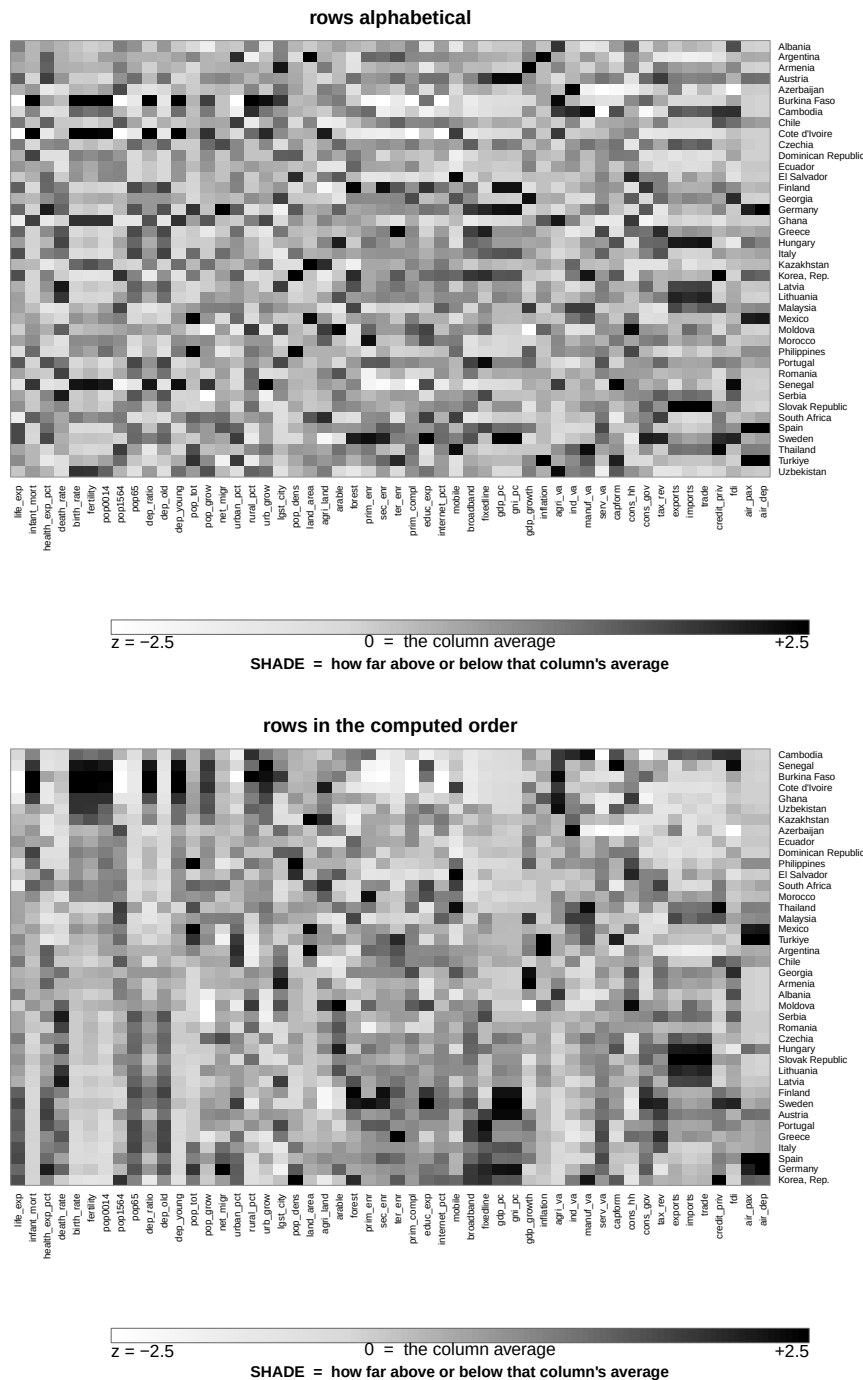
Fact 8.1: What the “computed order” was computed FROM (continued)

- **So the bands are not classes anybody found.** They are what adjacency looks like when adjacency was chosen to keep similar rows together. **The display did not discover “Europe”; it was handed a number and asked to sort.**

Now the same display at full size — 40 countries by 52 indicators, the 2080 cells nobody can read one at a time:

```
X52 <- w[, 5:56]
keep <- complete.cases(X52)           # TRUE where a row has no NA at all
Z52 <- scale(X52[keep, fam])          # fam: the codebook order of L4 5.3
rownames(Z52) <- w$country[keep]
Z52[Z52 > 2.5] <- 2.5; Z52[Z52 < -2.5] <- -2.5  # clip the extremes
dim(Z52)
```

```
[1] 40 52
```



Columns in codebook order in both panels; **only the rows differ**. Top: alphabetical — 2080 cells of noise.
Bottom: the same rows in the computed order.

Fact 8.2: Read the bottom panel

Three bands, top to bottom.

- **A low-income band opens it** — the *first five rows*, Cambodia, Senegal, Burkina Faso, Côte d'Ivoire, Ghana — averaging $z = +1.96$ on *fertility* and $z = -1.68$ on *internet_pct*: dark down the left of the map, pale down the right.
- **A middle-income band follows** — Ecuador, the Dominican Republic, the Philippines, El Salvador, South Africa... — sitting near $z = 0$ on the health and demographic columns and only mildly below average on income, which is what “middle” looks like.
- **A European band closes it** — the *last fourteen rows*, Czechia through Korea — the photographic negative of the first: $z = -0.65$ on *fertility*, $z = +1.12$ on *gdp_pc*.

Nobody told the algorithm that “Europe” exists — it was handed 40 rows and a dissimilarity, and the bands are what came out.

△! Two things this display does to you if you let it

1. The ready-made functions reorder both margins for you. We drew these with `image()`, which paints the matrix and nothing else. Reach instead for `stats::heatmap()` — or any package version of it — and it will **cluster your rows and your columns**, draw two trees, and hand back a picture whose ordering you did not choose and cannot defend. `heatmap(Z, Rowv = NA, Colv = NA)` switches that off; every package version has the same switch behind a different argument name. (What the clustering itself does is Lecture 26.) **An ordering that changes what the reader concludes must be one you can name.** And note the second reason we drew our own: `heatmap()` prints **no key** — there is no argument that adds one.

2. One extreme cell owns the greyscale. Without the clipping at ± 2.5 , Monaco's `gdp_pc` or the Central African Republic's `infant_mort` takes the whole dark end and everything else collapses to one mid-grey. Clipping is a defensible choice; *hiding* it is not, which is why it is on the printed line.

8.3 The whole gallery on one page

Display	Margin	Works while	What it hides
<code>ggpairs()</code>	col	$p \leq 8$, any n	everything that is not a property of a pair
<code>ggpairs(colour=)</code>	col	the groups are labelled	nothing extra — but the labels must exist
<code>corrplot()</code>	col	any p , order it	every unit, all curvature, all grouping
<code>stars()</code>	row	$n \leq 40$	every pair; and the ray order is your choice
<code>faces()</code>	row	never, in a report	as above, plus unequal feature prominence <i>and</i> variables drawn twice
<code>image()</code>	both	n, p both printable	nothing — but unreadable until <i>both</i> margins are ordered; <code>heatmap()</code> draws the same picture, orders both margins for you, and prints no key

Fact 8.3: The one sentence to keep from the appendix

Every display in this course is a choice of margin plus a choice of order, and neither choice is in the data. State both, or your reader is looking at your typing rather than at the world.

What is coming, so that you know what this appendix is standing in for

Three things above were used before the course has defined them, and each is deliberate. `dist()` and “**how unlike two countries are**” is Thursday — everything here that needed an ordering needed that number first, which is exactly why it is Thursday’s lecture and not this one. **Finding a good ordering of the *columns* from the data** rather than from the codebook is Lecture 21. **Turning an ordering into named groups** — what the bands in the heat map are — is Lecture 26. **Nothing here is examinable before those lectures arrive**; what *is* expected now is that you have read it and can say which display serves which margin.

For the course project

When you meet your own table in November, the first fifteen minutes are: `ggpairs()` if $p \leq 8$; `corrplot()` on an ordering you can justify if p is larger; `stars()` if there are only a few dozen units; the data heat map with both orderings stated. **Report what you chose and why** — a report that shows a heat map without saying how its rows were ordered has shown you a decision, not the data.

Ask an AI for this code — and check it

The four habits, in one place. Every box in these notes is the same four moves, and they are what this course expects of you when you use an assistant — in the labs, in the project, and afterwards.

- 1. Name the data and the constraint.** “My data frame `wh`, these four columns, base R only, no significance stars.” A vague prompt gets you `iris` and a package you do not have.
- 2. Ask for the tool the course uses.** Otherwise you get whatever is most common on the internet, which is usually the tidyverse, and your script stops matching everyone else’s.
- 3. Run it, and check one number you already know.** Every box above names that number: 6.52%, 0.26, three named species. *Code you cannot check against something you already believe is not evidence.*
- 4. Ask what it hides.** A recommendation with no cost attached is not advice — and the cost is what this whole lecture has been about.

And the one thing an assistant cannot do for you: **it does not know which margin your question is on** — and deciding that is your job.